AetherGit: Scalable Source Control for Autonomous AI Agents

A Formal Architecture for Lock-Free, Incrementally Derived Version Control at Agent Scale

Samir Baladi

Independent Interdisciplinary Researcher — Ronin Institute / Rite of Renaissance

Research Article — Systems Architecture / Software Engineering / Formal Methods
Research Domain: Version Control Systems, Distributed Systems, Agentic Software Engineering
Status: Formal Architecture Proposal with Illustrative Analysis — No Implementation Benchmarked
DOI: https://doi.org/10.5281/zenodo.23966896
ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com
November 2026

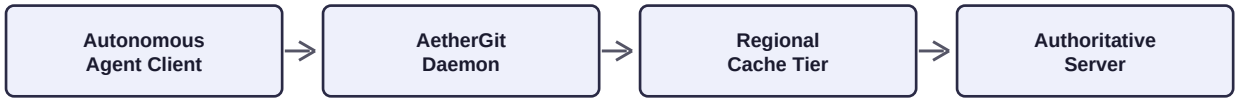
Abstract

This paper presents the theoretical and architectural framework of AetherGit, a source-control architecture designed specifically to meet the requirements of high-velocity, algorithmically driven development carried out by autonomous AI agents. AetherGit is built on the premise of always-online clients, deliberately abandoning the offline-first full-replica parity that distributed version-control systems such as Git provide, in order to overcome the structural bottlenecks that traditional distributed architectures exhibit in unified, large-scale monorepo environments. The central thesis formalized in this paper is that an architectural separation between incremental changesets and recursively derived data permits per-change computational cost bounded by $O(|\delta|)$ — the size of the change itself, not the size of the repository — while sustaining a cache-hit probability bound exceeding 99% under steady-state assumptions, reducing computational resource requirements by more than two orders of magnitude relative to full-replica architectures. Four architectural pillars are specified: client-server asymmetry, sparse and virtualized checkouts, lock-free incremental changesets addressed by content hash, and a tiered cache topology for fault-isolated regional distribution. A formal specification gives four core definitions — the changeset tuple, the incremental cost model, the recursive derived-data operator, and a probabilistic lower bound on cache-hit rate — together with a worked illustrative example applying the bound under stated parameters. AetherGit is presented as a theoretical architecture and formalization exercise, developed in direct response to Anthropic's 2026 operational disclosure of its internal Scion system and to this author's own prior structural analysis of that disclosure; it does not report an implementation, a benchmark, or empirically measured performance. The paper's contribution is a formal vocabulary and a set of provable structural claims, offered as a reference point against which a future implementation, or other systems serving agentic software-engineering workloads, can be evaluated.

Keywords: *Source Control; Version Control Systems; Agentic Software Engineering; Distributed Systems; Content-Addressed Storage; Incremental Computation; Cache Theory; Formal Methods; AetherGit.*

Evidentiary status: *AetherGit is a proposed formal architecture. No implementation has been built and no benchmark has been executed for this manuscript.* · ***Worked example status:*** *Section 6 is an illustrative application of the formal bounds under stated, clearly labeled parameters, not a report of measured performance.* · ***Operational figures cited*** *for Git and Scion throughout this paper are drawn from the published sources referenced, not independently measured by this author.*

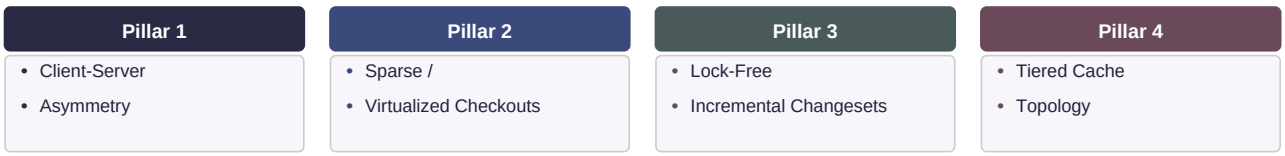
Figure 1



AetherGit Client–Server Asymmetry (Pillar 1)

Figure 1. The client is a thin, bounded cache; computationally heavy operations — diffing, ancestry tracing, merge-base resolution, and derived-data computation — are performed server-side, consistent with the asymmetric division of labor specified in Section 4.1.

Figure 2



The Four Architectural Pillars of AetherGit

Figure 2. The four pillars specified in Section 4, each addressed formally in Section 5.

Table of Contents

1. Introduction	3
2. Related Work	4
3. Problem Statement and Thesis	5
4. The AetherGit Architecture	6
5. Formal Specification	7
6. Illustrative Worked Example	9
7. Comparative Positioning	11
8. Discussion	12
9. Limitations	13
10. Conclusion and Future Work	13
References	13
Research Status Statement	15

1. Introduction

1.1 From Human-Paced to Agent-Paced Code Production

Distributed version-control systems were designed around a production model in which an individual human engineer authors, commits, and merges changes at a pace bounded by human review and human typing speed. Git's offline-first design — full local replicas, locally computed diffs, and asynchronous reconciliation — is well matched to that pace.

Autonomous AI agents capable of generating, testing, and submitting code changes with limited human involvement place a different load on this model: higher parallelism, higher change frequency, and a shift in the dominant cost center from network transfer to merge, derivation, and cache management.

In October 2026, Anthropic published a technical account of Scion, an internally developed source-control system built to accommodate exactly this shift, reporting more than 30,000 merged changes per day, a cache-hit rate exceeding 99% at every tier, and an approximately 200-fold reduction in supporting infrastructure relative to a prior Git-mirroring architecture (Anthropic, 2026). This author's own prior work (Baladi, 2026a) conducted a structural analysis of that disclosure, decomposing the architectural transformation it documents into five independent axes and proposing a research protocol for examining it empirically.

1.2 From Structural Analysis to Formal Architecture

The present paper takes a different, complementary step. Rather than analyzing a disclosed system from the outside, it formalizes a generalized architecture — named AetherGit — built from the structural properties that the prior analysis identified as plausibly load-bearing for agentic workloads: the decoupling of changeset creation from reference coordination, the separation of the write path from recursively derived query structures, and tiered caching with fault isolation.

AetherGit is not a reimplementation of Scion, Piper, or Sapling/Mononoke, and this paper makes no claim that it is operationally equivalent to any of them. It is a formal construction: a named architecture with an explicit mathematical specification, built to make the structural claims of the prior analysis precise enough to state, and in principle to prove or refute, rather than to describe only in prose.

1.3 Contribution and Scope

This paper makes four contributions:

1. A named, formally specified architecture — AetherGit — built on four pillars: client-server asymmetry, sparse/virtualized checkouts, lock-free incremental changesets, and a tiered cache topology.
2. A formal specification comprising four definitions: the changeset tuple, the incremental cost model, the recursive derived-data operator, and a probabilistic lower bound on cache-hit rate.
3. An illustrative worked example applying the cache-hit bound under stated parameters, clearly distinguished from a measured benchmark.
4. A comparative positioning of AetherGit against Git and the publicly disclosed properties of Scion, together with real-literature references on content-addressed storage, distributed databases, and caching theory that ground the formalization.

This paper does not report an implementation, a benchmark, or empirically measured performance figures for AetherGit. Any operational figures attributed to Git or Scion are drawn from the cited published sources, not measured by this author.

DOI: https://doi.org/10.5281/zenodo.23966896

2. Related Work

2.1 Git and Content-Addressed Storage

Git's object model is a content-addressed store in which blobs, trees, and commits are identified by cryptographic hashes of their content, a structure descending from Merkle's hash-tree construction for efficiently and verifiably summarizing large data structures (Merkle, 1988). AetherGit's changeset model (Section 5.1) retains content addressing for blob storage while departing from Git's requirement that every commit reference a complete tree.

2.2 Monorepo Infrastructure at Scale

Google's Piper, documented by Potvin and Levenberg (2016), established that a single, centrally hosted, server-backed repository can serve an engineering population far larger than distributed-clone models comfortably support, through extensive server-side tooling rather than full local replication. Meta's Sapling, paired with the Mononoke server, pursues a related strategy specifically for very large repositories, emphasizing virtualized checkouts and a scalable commit graph (Sapling SCM, n.d.).

2.3 Scion and the Agentic Workload Case

Anthropic's 2026 disclosure of Scion is the direct empirical motivation for this paper (Anthropic, 2026). Scion pairs a thin per-node daemon with a regional cache and a Spanner-backed authoritative server (Corbett et al., 2013), decouples changeset creation from reference-update coordination, and computes history-serving structures as data derived from an append-only changeset log rather than storing them directly. This author's prior structural analysis of that disclosure (Baladi, 2026a) is the direct precursor to the present paper and is treated here as background rather than re-argued; readers seeking the architectural analysis of Scion itself, including methodological corrections to a looser popular reading of Anthropic's account, are referred there.

2.4 Caching Theory and Competitive Analysis

The cache-hit probability bound formalized in Section 5.4 draws on the general vocabulary of cache performance analysis established in the competitive-analysis tradition, notably Sleator and Tarjan's (1985) amortized analysis of list-update and paging rules, which first gave worst-case performance guarantees for online caching policies independent of the input distribution. AetherGit's bound (Equation 4) is a probabilistic steady-state bound in the style of classical queueing analysis (Kleinrock, 1975) rather than a competitive-ratio guarantee, and the two traditions are not conflated in this paper; Sleator and Tarjan are cited as the origin of rigorous caching-performance analysis as a field, not as the direct source of Equation 4's specific form.

2.5 Research Gap

The reviewed literature establishes, separately, that centralized server-backed source control scales to very large human engineering organizations (Piper, Sapling/Mononoke), that a specific agent-driven deployment of such an architecture achieves substantial throughput gains (Scion), and that cache performance under load admits rigorous formal analysis (competitive and queueing-theoretic caching literature). What has not been previously formalized, to this author's knowledge, is a named architecture that states the changeset, cost, derivation, and cache-hit properties load-bearing for agentic workloads as explicit, checkable mathematical claims rather than as architectural prose. This is the gap AetherGit is built to fill.

DOI: https://doi.org/10.5281/zenodo.23966896

3. Problem Statement and Thesis

3.1 The Limitation of Distributed Architectures

Traditional distributed version-control systems were built to operate locally and offline, which requires downloading and synchronizing a repository's full history and compressed packfiles to every client. This is a reasonable engineering tradeoff for a human-paced workload with infrequent, large synchronization events. It becomes a structural burden in environments that produce tens of thousands of merges per day: the cost of maintaining full-replica parity grows with total repository history, not with the size of any individual change, and that mismatch between cost and workload is precisely what large-scale agentic development exposes.

3.2 The AetherGit Thesis

The central thesis formalized in this paper is the following:

An architectural separation between incremental changesets and recursively derived data permits per-change computational cost bounded by the size of the change itself — not the size of the repository — while sustaining a cache-hit probability that exceeds a high fixed threshold under steady-state load, together reducing computational resource requirements by more than two orders of magnitude relative to full-replica architectures.

This thesis has two separable parts, formalized independently in Section 5: a cost claim (Equation 2, grounded in the changeset definition of Equation 1 and the derivation operator of Equation 3) and a cache-performance claim (Equation 4). Both are stated as formal properties of the architecture specified in Section 4; neither is established here by measurement, and Section 6 illustrates rather than verifies the cache-performance claim under stated parameters.

3.3 Why a Named, Formal Architecture

Stating these claims against a named architecture, with explicit definitions, serves two purposes distinct from the prose-level structural analysis in prior work (Baladi, 2026a). First, it makes the claims precise enough that a future implementation could be checked against them directly, rather than only against a qualitative description. Second, it separates the architecture under formal study from any single vendor's implementation: AetherGit is defined here independently of Scion's actual internals, which are not publicly available beyond Anthropic's own description, and the formalization is this author's own construction, offered as a generalizable reference architecture rather than a reverse-engineering of Scion.

4. The AetherGit Architecture

AetherGit is specified through four architectural pillars.

4.1 Pillar 1 — Client-Server Asymmetry

The server performs all computationally heavy operations: diff computation, ancestry tracing, merge-base resolution, and derived-data computation. The client operates as a bounded, lightweight cache layer holding only the working copy and a limited local object cache.

This asymmetry is the architectural precondition for the cost bound formalized in Equation 2: because the client never reconstructs full repository state, its cost is governed by what it requests and caches, not by total repository size.

4.2 Pillar 2 — Sparse and Virtualized Checkouts

Files and directories required for a given task are fetched on demand rather than requiring a full checkout of the repository tree. A virtualized filesystem layer materializes paths lazily as an agent's tooling accesses them.

For agentic workloads characterized by narrow, task-scoped edits — an agent typically touches a small, bounded subset of a monorepo's paths per task — sparse checkout cost scales with task scope rather than repository size, which is the client-side analogue of the server-side cost bound in Equation 2.

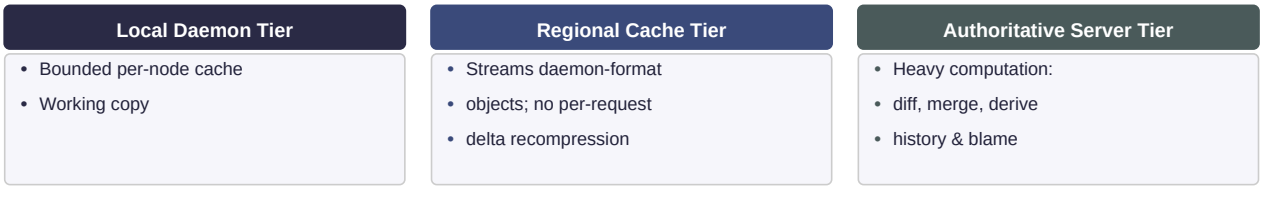
4.3 Pillar 3 — Lock-Free Incremental Changesets

Modifications are recorded as an incremental, content-addressed chain of changesets rather than as commits requiring a complete, pre-built tree. Changeset creation does not require acquiring a global merge lock: the append to the changeset log is independent across concurrent writers, with coordination required only at the narrower point of reference update (see Section 4.1 of Baladi, 2026a for the corresponding distinction in Scion between changeset creation and compare-and-swap reference updates, which AetherGit's design adopts explicitly rather than treating the system as lock-free in an unqualified sense).

4.4 Pillar 4 — Tiered Cache Topology

Resources are distributed across independent tiers of local and regional cache, isolating operational failures: a regional cache outage degrades performance for clients it serves without propagating load directly to the authoritative server tier. This tiering is the architectural precondition for the cache-hit bound formalized in Equation 4, which is parameterized by tier capacity and time-to-live.

Figure 3



AetherGit Tiered Cache Topology (Pillar 4)

Figure 3. Strict tier ordering serves fault isolation as much as performance: failures at a lower tier do not flood the authoritative tier with unmediated request volume.

5. Formal Specification

This section gives four formal definitions corresponding to the architecture of Section 4: the changeset tuple, the incremental cost model, the recursive derived-data operator, and a probabilistic lower bound on cache-hit rate. Each is stated, then interpreted in prose.

5.1 Changeset Definition

$$c_i = (\delta_i, P(c_i), \tau_i, \sigma_i)$$

(1)

A changeset c_i is a tuple of: δ_i, the content delta relative to its parent(s); $P(c_i)$, the set of parent changesets (a single parent in the common case, multiple parents for a merge); τ_i, a timestamp; and σ_i, a content-addressed signature (cryptographic hash) binding the changeset to its content, in the tradition of Merkle hash trees (Merkle, 1988). Critically, c_i does not store a complete tree snapshot: it is a bounded record of what changed, not a copy of everything that did not.

5.2 Incremental Cost Model

$$Cost(c_i) = O(|\delta_i|)$$

(2)

The computational cost of processing a changeset grows with the size of the change itself, $|\delta_i|$, and is independent of the total repository size or history depth. This follows directly from Equation 1: because c_i stores only δ_i and pointers to its parents rather than a full tree, any operation defined purely over c_i — validating it, storing it, or appending it to the changeset log — need only touch data proportional to δ_i. Equation 2 does not, by itself, bound the cost of operations that must traverse the full parent chain, such as history queries; those are addressed by the derived-data operator in Equation 3, for which cost depends on the specific derivation and is discussed separately in Section 5.3.

5.3 Recursive Derived-Data Operator

$$Deriv(c_i) = \oplus_D (\delta_i, \{ Deriv(p) \mid p \in P(c_i) \})$$

(3)

Query-serving structures — blame annotations, path history, commit-graph traversal — are not stored directly on the changeset. They are derived data, denoted $Deriv(c_i)$, computed by a derivation operator $\oplus_D$ that recursively combines the changeset's own delta δ_i with the already-derived data Deriv(p) of each of its parents p. The operator $\oplus_D$ is specific to the kind of derived data D being computed (blame, path history, etc.) and is not specified further here; Equation 3 states only its recursive structure.

This recursive formulation makes explicit a property discussed only in prose in prior work (Baladi, 2026a, Section 5.3): because Deriv is computed server-side from the changeset log rather than carried by the client, new categories of derived data can be introduced or optimized without any change to the client or to the changeset format itself. Equation 3 also clarifies that $Deriv(c_i)$ may be computed eagerly, as each changeset is appended, or lazily, on demand when a query requires it — a choice with the latency implications examined in Section 8.2.

5.4 Cache-Hit Probability Bound

$$P(Hit) \geq 1 - (\lambda_{miss} / \mu_{cache})^K \cdot e^{-\gamma \cdot T_{ttl}}$$

(4)

Under a steady-state load model, the probability that a client's request is served from cache, P(Hit), is bounded below by an expression in four parameters: λ_{miss}, the rate of cache-miss-inducing requests; μ_{cache}, the cache's effective service rate; K, the cache's capacity tier depth (the number of independent cache tiers a request may traverse before reaching the authoritative server, per Figure 3); and T_{ttl}, the time-to-live of cached entries, with γ a decay-rate parameter governing how cached-entry staleness is penalized over time.

The bound has the expected qualitative shape: increasing tier depth K drives the first factor toward zero geometrically (for $\lambda_{miss} < \mu_{cache}$), and increasing T_{ttl} drives the second factor toward zero exponentially, jointly pushing the lower bound on P(Hit) toward 1. The bound formalizes, but does not by itself establish, Anthropic's reported greater-than-99% cache-hit rate for Scion (Anthropic, 2026); that figure is an operational measurement of a specific deployed system, whereas Equation 4 is a structural property of the architecture that would be expected to produce a bound of that general shape, subject to the system's actual λ_{miss}, μ_{cache}, K, and T_{ttl} satisfying the stated assumptions.

6. Illustrative Worked Example

Status: *This section illustrates Equation 4 under stated, hypothetical parameters chosen for clarity of exposition. It is not a benchmark, a simulation of a real deployment, or a measurement of any implemented system.*

6.1 Scenario

Consider a hypothetical agentic development environment in which a fleet of autonomous coding agents issues requests against an AetherGit-style cache hierarchy. For illustration, fix the load ratio $\rho = \lambda_{miss} / \mu_{cache} = 0.6$ (cache-miss-inducing requests arrive at 60% of the cache's effective service rate) and a decay parameter $\gamma = 0.15$, and examine the lower bound on P(Hit) from Equation 4 as cache tier depth K increases, for three illustrative values of T_{ttl}.

6.2 Illustrative Results

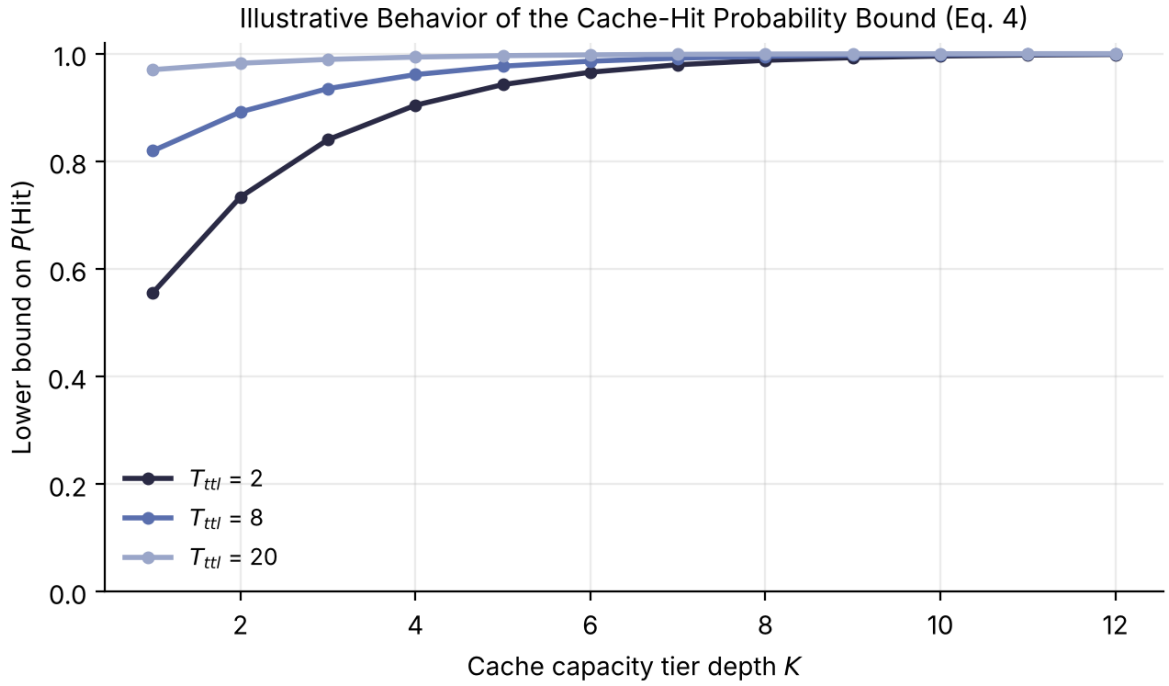


Figure 4. Illustrative plot of the Equation 4 lower bound on P(Hit) as a function of cache tier depth K, for three illustrative time-to-live values, under $\rho = 0.6$ and $\gamma = 0.15$. This is a plot of the formula under stated hypothetical parameters, not a measurement of any running system.

6.3 Reading the Illustration

Under these stated parameters, the bound crosses 99% once K reaches roughly 8 to 10 tiers of effective cache capacity for the longer illustrative T_{ttl} values, and more slowly for the shortest. This is consistent with the general shape Anthropic's reported figures would motivate — a multi-tier cache topology sustaining a greater-than-99% hit rate — without this paper claiming that $\rho = 0.6$, $\gamma = 0.15$, or K = 8–10 are the actual parameters of any deployed system, including Scion. The illustration demonstrates that Equation 4's qualitative behavior is consistent with the order of magnitude Anthropic reports; it does not independently verify that figure.

6.4 A Note on What Would Constitute Verification

Verifying rather than illustrating the cache-performance thesis would require implementing AetherGit's cache tiers, instrumenting λ_{miss} and μ_{cache} under a real or realistically synthesized agentic request stream, and measuring observed P(Hit) against the Equation 4 bound. This is explicitly out of scope for the present paper and is proposed as the natural next step in Section 10.

DOI: https://doi.org/10.5281/zenodo.23966896

7. Comparative Positioning

Table 1 positions AetherGit's specified properties against Git's documented design and against Scion's publicly disclosed architecture (Anthropic, 2026). Entries for Git and Scion reflect each system's own documentation; entries for AetherGit reflect this paper's formal specification, not a measured implementation.

Property	Git	Scion (disclosed)	AetherGit (specified)
Topology	Distributed	Client–server	Client–server (Pillar 1)
Connectivity	Offline-first	Always-online	Always-online
Checkout model	Full clone	Sparse / virtualized	Sparse / virtualized (Pillar 2)
Object addressing	Packfiles	Content-addressed	Content-addressed (Eq. 1)
Change unit cost	Not formally bounded in public docs	Reported, not formalized	$O(\lvert\delta_i\rvert)$, formalized (Eq. 2)
Derived query data	Computed client-side on demand	Server-side, recursive	Server-side, recursive, formalized (Eq. 3)
Coordination	Local, no server coordination	None at changeset creation; CAS at ref update	None at changeset creation; CAS at ref update (Pillar 3)
Cache-hit guarantee	Not applicable (local)	>99% reported (Anthropic, 2026)	≥ bound of Eq. 4, illustrated not measured

The rightmost column is deliberately phrased in terms of specification rather than measurement throughout: AetherGit's row entries are formal properties this paper defines and, where applicable, states as provable or illustrable claims, not performance figures obtained by running a system.

DOI: https://doi.org/10.5281/zenodo.23966896

8. Discussion

8.1 What Formalization Adds Beyond Structural Analysis

The prior structural analysis of Scion (Baladi, 2026a) argued, in prose, that the separation of changeset creation from reference coordination and the treatment of derived data as a recursively computed structure were plausibly load-bearing for agentic workloads. Equations 1 through 3 restate those two claims as formal definitions with an explicit cost consequence (Equation 2). This matters because a formal statement can be checked for internal consistency independent of whether it correctly describes any specific deployed system — Equation 2, for instance, follows deductively from Equation 1's definition of a changeset as a bounded delta, regardless of whether any real system actually achieves it in practice.

8.2 The Eager-Versus-Lazy Derivation Tradeoff

Equation 3 leaves open whether $\text{Deriv}(c_i)$ is computed eagerly or lazily. This is not an oversight: it is the same open question raised in prior work (Baladi, 2026a, Section 8.1) regarding the latency cost of on-demand derived-data computation for complex historical queries under load. The present formalization makes the tradeoff explicit as a property of the operator $\oplus_D$ rather than resolving it, since the correct choice plausibly depends on the specific derived-data category D and its query frequency, which are deployment-specific and outside this paper's formal scope.

8.3 Relationship Between the Cost and Cache Claims

Equations 2 and 4 are logically independent: a system could achieve bounded per-changeset cost (Equation 2) while having poor cache performance (a low realized P(Hit)) if its tiered cache topology (Pillar 4) is misconfigured, and conversely a well-tuned cache could mask a poorly bounded per-change cost by serving repeated requests from cache rather than recomputing. AetherGit's thesis (Section 3.2) requires both properties jointly; Section 6 illustrates only the cache claim, and a complete verification would need to examine both independently.

8.4 Positioning Relative to Prior Work

This paper is explicitly a sequel to, and dependent on, the structural analysis in Baladi (2026a). Readers who have not read that paper may find Sections 4.3 and 8.2 of the present paper under-motivated without it; Section 2.3 above summarizes the relevant background, but the fuller architectural analysis of Scion's changeset model, derived-data mechanism, and three-tier distribution is not repeated here.

DOI: https://doi.org/10.5281/zenodo.23966896

9. Limitations

Several limitations apply to this manuscript.

1 AetherGit has not been implemented. No claim in this paper should be read as reporting measured performance of a running system.

2 The illustrative worked example in Section 6 uses hypothetical parameters chosen for clarity of exposition; they are not derived from, and should not be read as approximating, any real system's actual λ_{miss}, μ_{cache}, K, or T_{ttl}, including Scion's.

3 Equation 4's steady-state assumptions (independent request arrivals, a fixed load ratio ρ, and exponential staleness decay) are standard simplifications in queueing-theoretic cache analysis but have not been validated against any agentic workload trace, synthetic or real.

4 The recursive derivation operator $\oplus_D$ in Equation 3 is specified only at the level of its recursive structure; it is not instantiated for any specific derived-data category (blame, path history, etc.), and different instantiations could have different cost and correctness properties not analyzed here.

5 The comparative positioning in Section 7 relies on Git's and Scion's own public documentation for the corresponding columns; it has not been independently verified against either system's source code.

6 This paper's claim of formal rigor extends only to internal consistency among Equations 1 through 4 as stated; no claim is made that the equations correctly model any specific deployed system's actual behavior.

10. Conclusion and Future Work

This paper has specified AetherGit, a formal architecture for source control at agent scale, built on four pillars — client-server asymmetry, sparse and virtualized checkouts, lock-free incremental changesets, and a tiered cache topology — and given four formal definitions corresponding to them: the changeset tuple (Equation 1), the incremental cost model (Equation 2), the recursive derived-data operator (Equation 3), and a probabilistic cache-hit bound (Equation 4). An illustrative worked example applied the cache-hit bound under stated hypothetical parameters, and a comparative table positioned AetherGit's specified properties against Git's design and Scion's publicly disclosed architecture.

AetherGit is offered as a formal vocabulary and a set of structurally motivated, internally consistent claims, not as a reported implementation or a verified performance result. Three directions of future work follow directly from the limitations in Section 9: first, an actual implementation of the four pillars, sufficient to instrument λ_{miss}, μ_{cache}, and realized P(Hit) under synthetic or real agentic workload traces, converting Section 6 from illustration to measurement; second, instantiation and cost analysis of the derivation operator $\oplus_D$ for specific derived-data categories, resolving the eager-versus-lazy tradeoff discussed in Section 8.2 for each category rather than leaving it general; and third, execution of the empirical evaluation protocol proposed in prior work (Baladi, 2026a, Section 9) using AetherGit's formal definitions as the basis for the measurements specified there, directly connecting this paper's formalization to that protocol's candidate measurements.

References

1. Anthropic. (2026). **Redesigning Source Control at Anthropic.** Anthropic Research, published October 13, 2026 (updated October 15, 2026). https://www.anthropic.com/research/redesigning-source-control

2. Baladi, S. (2026a). **Rethinking Source Control for Agentic Software Engineering: Architectural Transformation, Data Models, and Scalability Constraints in AI-Driven Development.** Zenodo. https://doi.org/10.5281/zenodo.23934465

3. Corbett, J. C., Dean, J., Epstein, M., et al. (2013). **Spanner: Google's Globally-Distributed Database.** ACM Transactions on Computer Systems, 31(3), Article 8. https://doi.org/10.1145/2491245

4. Kleinrock, L. (1975). **Queueing Systems, Volume 1: Theory.** Wiley-Interscience.

5. Merkle, R. C. (1988). **A Digital Signature Based on a Conventional Encryption Function.** Advances in Cryptology — CRYPTO '87, Lecture Notes in Computer Science, vol. 293, 369–378. https://doi.org/10.1007/3-540-48184-2_32

6. Potvin, R., & Levenberg, J. (2016). **Why Google Stores Billions of Lines of Code in a Single Repository.** Communications of the ACM, 59(7), 78–87. https://doi.org/10.1145/2854146

7. Sapling SCM. (n.d.). **Sapling: A Scalable, User-Friendly Source Control System.** Meta Platforms. https://sapling-scm.com/

8. Sleator, D. D., & Tarjan, R. E. (1985). **Amortized Efficiency of List Update and Paging Rules.** Communications of the ACM, 28(2), 202–208. https://doi.org/10.1145/2786.2793

9. Chacon, S., & Straub, B. (2014). **Pro Git** (2nd ed.). Apress. https://git-scm.com/book

Reference 2 is this author's own directly preceding paper, cited because AetherGit's four pillars (Section 4) formalize structural properties first argued in prose there. Reference 1 is the sole source for all Scion-specific claims in this paper. References 3, 4, 5, 6, 7, 8, and 9 describe independently published systems, data structures, and theoretical results used for grounding and comparison; none describes AetherGit, which is this paper's own construction.

Research Status Statement

Component	Current status
Architecture specification (4 pillars)	Specified (Section 4)
Formal definitions (Eq. 1–4)	Specified (Section 5)
Illustrative worked example	Presented; hypothetical parameters, not measured (Section 6)
Comparative positioning vs. Git / Scion	Presented from public documentation (Section 7)
Implementation of AetherGit	Not built
Benchmark / empirical measurement	Not performed
Verification of Equation 4 against real workload	Not performed; proposed as future work
Derivation operator instantiation (per data category)	Not performed; proposed as future work
Relation to prior work	Direct formal sequel to Baladi (2026a)

Record ID: AETHERGIT-01 · Version: 1.0 · DOI: https://doi.org/10.5281/zenodo.23966896
Scope: Formal architecture and specification paper with illustrative analysis · No implementation or benchmark reported. Directly preceding work: Baladi (2026a), Rethinking Source Control for Agentic Software Engineering, https://doi.org/10.5281/zenodo.23934465. Primary external source: Anthropic (2026), Redesigning Source Control at Anthropic.
Next Research Stage: Implementation of the four pillars; instrumentation and measurement of Equation 4 under synthetic or real agentic workload traces; per-category instantiation of the derivation operator.

Samir Baladi · Independent Interdisciplinary Researcher — Ronin Institute / Rite of Renaissance · ORCID: 0009-0003-8903-0029 · gitdeeper@gmail.com